Lab 2 FLIP 流体仿真实现报告

1 基础 FLIP 算法实现

本次实现采用粒子和网格混合的 FLIP/PIC 方法。粒子负责保存流体的位置和速度，用于表达自由表面和进行平流；规则网格负责压力投影，用于消除速度场散度并近似满足不可压缩条件。每一帧仿真被拆成若干 substep，以降低大速度下粒子穿透边界和障碍物的概率。

单个时间步的流程如下：先对粒子速度施加重力并积分位置，然后处理容器边界和障碍物碰撞；之后通过粒子分离减少粒子重叠；接着将粒子速度传到 MAC 网格上，标记固体、空气和流体单元；在网格上进行 Gauss-Seidel 压力投影；最后把投影后的网格速度回传给粒子，并根据 `flipRatio` 在 PIC 和 FLIP 之间插值。

Listing 1: 单步 FLIP 仿真流程

```
void SimulateTimestep(float const dt) {
    float maxSpeed = glm::length(m_obstacleVel);
    for (glm::vec3 const & vel : m_particleVel) {
        maxSpeed = std::max(maxSpeed, glm::length(vel));
    }

    int numSubSteps = std::clamp(
        int(std::ceil(std::max(1.0f, dt * std::max(maxSpeed, 2.0f) / (0.45f *
            m_h)))),
        1, 6);
    float sdt = dt / numSubSteps;

    for (int step = 0; step < numSubSteps; step++) {
        integrateParticles(sdt);
        handleParticleCollisions(obstaclePos, obstacleRadius, obstacleVel);
        pushParticlesApart(numParticleIters);
        handleParticleCollisions(obstaclePos, obstacleRadius, obstacleVel);
        transferVelocities(true, flipRatio);
        updateParticleDensity();
        solveIncompressibility(numPressureIters, sdt, overRelaxation,
            compensateDrift);
        transferVelocities(false, flipRatio);
    }
    updateParticleColors();
}
```

1.1 粒子平流与边界处理

粒子速度首先受到重力影响，然后用显式欧拉方法更新位置。为了避免极端速度导致数值爆炸，我对速度做了上限限制。容器边界被限制在模拟盒内部，粒子碰到边界时对应方向速度置零。

Listing 2: 粒子积分与速度限制

```
void Simulator::integrateParticles(float timeStep) {
    float const maxVelocity = 3.0f / std::max(m_h, kEps);

    for (int i = 0; i < m_iNumSpheres; ++i) {
        m_particleVel[i] += gravity * timeStep;

        float const speed = glm::length(m_particleVel[i]);
        if (speed > maxVelocity) {
            m_particleVel[i] *= maxVelocity / speed;
        }

        m_particlePos[i] += m_particleVel[i] * timeStep;
    }
}
```

边界和障碍物碰撞采用位置修正加速度投影的方法。若粒子进入障碍物内部，先把粒子推出到障碍物表面，再去掉相对障碍物速度中朝向障碍物内部的法向分量。这样障碍物移动时可以把速度传递给附近粒子。

Listing 3: 球形障碍物的碰撞处理

```
glm::vec3 delta = pos - obstaclePos;
float dist2 = glm::dot(delta, delta);
float minDist = obstacleRadius + m_particleRadius;
if (dist2 < minDist * minDist) {
    float dist = std::sqrt(std::max(dist2, kEps));
    glm::vec3 normal = dist > kEps ? delta / dist : glm::vec3(0.0f, 1.0f, 0.0f
        );
    pos = obstaclePos + normal * minDist;

    glm::vec3 relVel = vel - obstacleVel;
    float vn = glm::dot(relVel, normal);
    if (vn < 0.0f) {
        relVel -= vn * normal;
    }
    vel = relVel + obstacleVel;
}
```

1.2 粒子到网格与 PIC/FLIP 回传

速度传递使用三线性权重。粒子到网格时，把每个粒子的三个速度分量分别累加到对应方向的网格面速度上，同时记录权重和，最后做归一化。网格单元会根据固体 mask 和是否包含粒子被标

记为 SOLID_CELL、EMPTY_CELL 或 FLUID_CELL。

网格回传粒子时同时计算 PIC 速度和 FLIP 速度。PIC 直接插值当前网格速度，数值耗散较大但稳定；FLIP 则把网格速度变化量叠加回粒子原速度，能保留更多涡旋和动量。最终用 flipRatio 做线性混合：

$$v_p^{new} = (1-r)v_{PIC} + rv_{FLIP}.$$

因此 flipRatio = 0 是 PIC，flipRatio = 1 是纯 FLIP，flipRatio = 0.95 是常用的 FLIP95。

Listing 4: PIC/FLIP 混合回传

```
forEachFaceWeight(*this, m_particlePos[p], dir, [&](int gridOffset, float
    weight) {
    pic += component(m_vel[gridOffset], dir) * weight;
    flip += (component(m_vel[gridOffset], dir)
            - component(m_pre_vel[gridOffset], dir)) * weight;
    wsum += weight;
});

if (wsum > 0.0f) {
    component(m_particleVel[p], dir) =
        (1.0f - flipRatio) * pic + flipRatio * flip;
}
```

1.3 不可压缩投影

不可压缩部分在网格上使用 Gauss-Seidel 迭代。对每个流体单元计算散度，然后根据周围可通行方向数量求出本次压力修正量，并直接修改相邻面速度。每轮迭代后重新应用边界速度，保证固体边界处不会产生穿透速度。

实现中在投影前保存 m_pre_vel，用于后续 FLIP 回传时计算网格速度变化量。同时加入了基于粒子密度的 drift compensation：如果局部粒子密度高于静止密度，则额外减少散度，缓解流体体积压缩。

Listing 5: Gauss-Seidel 压力投影

```
float div =
    m_vel[index2GridOffset({ i + 1, j, k })].x - m_vel[index2GridOffset({ i, j
        , k })].x +
    m_vel[index2GridOffset({ i, j + 1, k })].y - m_vel[index2GridOffset({ i, j
        , k })].y +
    m_vel[index2GridOffset({ i, j, k + 1 })].z - m_vel[index2GridOffset({ i, j
        , k })].z;

if (compensateDrift && m_particleRestDensity > 0.0f) {
    float const compression = m_particleDensity[idx] - m_particleRestDensity;
    if (compression > 0.0f) {
        div -= 0.8f * compression;
    }
```

```
}

float const pressureDelta = -overRelaxation * div / s;
m_p[idx] += pressureScale * pressureDelta;

m_vel[index2GridOffset({ i, j, k })].x     -= sx0 * pressureDelta;
m_vel[index2GridOffset({ i + 1, j, k })].x += sx1 * pressureDelta;
m_vel[index2GridOffset({ i, j, k })].y     -= sy0 * pressureDelta;
m_vel[index2GridOffset({ i, j + 1, k })].y += sy1 * pressureDelta;
m_vel[index2GridOffset({ i, j, k })].z     -= sz0 * pressureDelta;
m_vel[index2GridOffset({ i, j, k + 1 })].z += sz1 * pressureDelta;
```

1.4 交互参数

界面中提供了时间步长和 FLIP 比例两个核心参数。时间步长控制每帧推进的物理时间；`FLIP Ratio` 控制 PIC/FLIP 混合比例，可以直接观察 PIC 更稳定但更耗散、FLIP 更活跃但更容易产生噪声的差异。

Listing 6: 基础交互参数

```
ImGui::SliderFloat("Time Step", &_timeStep, 0.001f, 0.05f, "%.4f");
ImGui::SliderFloat("FLIP Ratio", &_flipRatio, 0.0f, 1.0f, "%.2f");
ImGui::Text("FLIP Ratio: 0.0 = PIC, 1.0 = FLIP, 0.95 = FLIP95");
```

2 Bonus 1：可视化与交互增强

Bonus 1 中我实现了两部分：第一是按粒子速度大小染色，第二是可拖动障碍物与流体交互。

粒子颜色使用速度归一化后的结果在深蓝和浅蓝之间插值。静止或低速区域为深蓝色，高速区域逐渐变成浅蓝色。为了让视觉区分更明显，我对归一化速度做了放大和幂函数调整，使中等速度也能产生可见变化。

Listing 7: 按速度大小进行蓝色系染色

```
void Simulator::updateParticleColors() {
    float maxSpeed = 1e-4f;
    for (int i = 0; i < m_iNumSpheres; ++i) {
        maxSpeed = std::max(maxSpeed, glm::length(m_particleVel[i]));
    }

    for (int i = 0; i < m_iNumSpheres; ++i) {
        float const speed01 = std::clamp(
            1.35f * glm::length(m_particleVel[i]) / maxSpeed, 0.0f, 1.0f);
        float const tint = 0.08f + 0.72f * std::pow(speed01, 0.65f);
        glm::vec3 const deepBlue(0.02f, 0.08f, 0.38f);
        glm::vec3 const lightBlue(0.32f, 0.56f, 0.92f);
        m_particleColor[i] = glm::mix(deepBlue, lightBlue, tint);
```

```
    }
}
```

交互障碍物支持球体和立方体两种形状，可以在 UI 中切换。按住 `Z` + 鼠标左键并点中障碍物后，可以在屏幕平面内拖动障碍物。每帧根据障碍物位置变化计算障碍物速度，再传入流体求解器。粒子碰到移动障碍物时会使用相对速度进行碰撞响应，因此可以看到障碍物推动液体的效果。

Listing 8: 障碍物 UI 与拖动说明

```
int obstacleType = static_cast<int>(_obstacleShape);
if (ImGui::Combo("Obstacle Type", &obstacleType, "Sphere\0Cube\0")) {
    _obstacleShape = static_cast<ObstacleShape>(obstacleType);
    _simulation.setObstacle(_obstacleShape, _obstaclePos,
                            glm::vec3(0.0f), _obstacleRadius, true);
}
ImGui::Text("Z + Left Drag On Obstacle: move obstacle");
```

Listing 9: 由鼠标拖动计算障碍物位移

```
glm::vec3 deltaWorld =
    right * (io.MouseDelta.x * heightNorm * panScale) +
    _sceneObject.Camera.Up * (-io.MouseDelta.y * heightNorm * panScale);

_obstaclePos += deltaWorld;
float const bound = 0.5f - 2.0f * _simulation.m_h - _obstacleRadius;
_obstaclePos.x = std::clamp(_obstaclePos.x, -bound, bound);
_obstaclePos.y = std::clamp(_obstaclePos.y, -bound, bound);
_obstaclePos.z = std::clamp(_obstaclePos.z, -bound, bound);
_obstacleDraggedThisFrame = true;
```

3 Bonus 5：简单表面重建与渲染

Bonus 5 中我实现了一个可开关的流体表面渲染选项。由于完整的高质量 Marching Cubes 表面重建代码量较大，我采用了一个简化方案：先把粒子密度累积到网格上，使用静止密度归一化，得到一个近似的标量场；然后在细分后的网格上对密度做三线性插值；最后对每个小立方体做 tetrahedra 分解，在四面体内部根据等值面阈值提取三角形。该方法本质上是 Marching Tetrahedra 风格的简化等值面提取，能够生成连续的液体表面 mesh。

UI 中使用 `Mesh` 复选框控制是否显示表面。开启后可以通过 `Mesh Detail` 调整重建网格细分倍率，细节越高，表面越细腻，但生成三角形数量和渲染开销也更大。

Listing 10: 表面重建 UI

```
ImGui::Checkbox("Mesh", &_showMesh);
if (_showMesh) {
    ImGui::SliderInt("Mesh Detail", &_meshDetail, 1, 4);
}
```

表面重建首先构造细网格采样。采样值来自原始粒子密度网格的三线性插值，并除以静止密度归一化。等值面阈值设置为 `0.55f`，表示密度达到一定比例的位置被认为在流体内部。

Listing 11: 密度场三线性采样

```
float const restDensity = std::max(_simulation.m_particleRestDensity, 1e-4f);
float const isoValue = 0.55f;

auto coarseDensityAt = [&](int i, int j, int k) {
    i = std::clamp(i, 0, nx - 1);
    j = std::clamp(j, 0, ny - 1);
    k = std::clamp(k, 0, nz - 1);
    return _simulation.m_particleDensity[
        _simulation.index2GridOffset(glm::ivec3(i, j, k))] / restDensity;
};

auto densityAt = [&](int i, int j, int k) {
    float gx = float(i) / float(refine);
    float gy = float(j) / float(refine);
    float gz = float(k) / float(refine);
    ...
    return glm::mix(c0, c1, tz);
};
```

每个细网格 cube 被划分成 6 个 tetrahedra。对每个四面体统计顶点在等值面内外的数量：若为 1/3 分布则生成一个三角形，若为 2/2 分布则生成两个三角形。顶点位置通过线性插值得到，使三角形落在密度等值面上。

Listing 12: 四面体等值面提取

```
auto interpolate = [&](glm::vec3 const & pa, float va,
                       glm::vec3 const & pb, float vb) {
    float const denom = std::abs(vb - va) > 1e-6f ? (vb - va) : 1e-6f;
    float const t = std::clamp((isoValue - va) / denom, 0.0f, 1.0f);
    return glm::mix(pa, pb, t);
};

if (insideCount == 1 || insideCount == 3) {
    int const apex = insideCount == 1 ? inside[0] : outside[0];
    int const * rim = insideCount == 1 ? outside : inside;
    glm::vec3 const a = interpolate(nodePos[apex], nodeValue[apex],
                                    nodePos[rim[0]], nodeValue[rim[0]]);
    glm::vec3 const b = interpolate(nodePos[apex], nodeValue[apex],
                                    nodePos[rim[1]], nodeValue[rim[1]]);
    glm::vec3 const c = interpolate(nodePos[apex], nodeValue[apex],
                                    nodePos[rim[2]], nodeValue[rim[2]]);
    emitTriangle(positions, indices, a, b, c);
}
```

渲染时，如果 Mesh 打开，则绘制重建出的三角网格；否则仍然显示粒子球。这样可以在同一套仿真数据上切换粒子视图和表面视图。

Listing 13: 粒子视图和表面视图切换

```
if (_showMesh) {
    UpdateSurfaceMesh();
}

if (_showMesh) {
    _surfaceItem.Draw({ _surfaceProgram.Use() });
} else {
    Rendering::ModelObject m =
        Rendering::ModelObject(_sphere, _simulation.m_particlePos,
                               _simulation.m_particleColor);
    m.Mesh.Draw({ _program.Use() },
        _sphere.Mesh.Indices.size(), 0, _simulation.m_iNumSpheres);
}
```